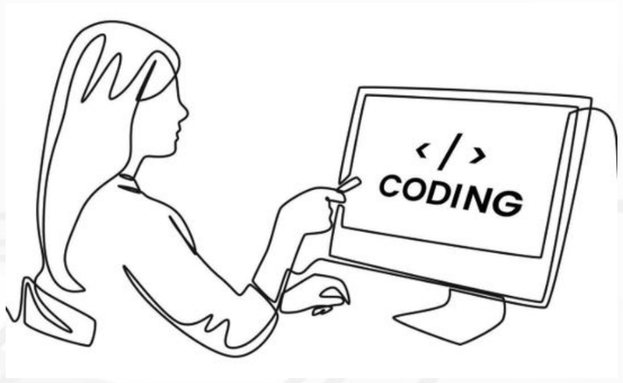
FADENA
FACULTAD DE DEFENSA NACIONAL

Lenguajes de Programación

Clase 2: Pseudocódigo – parte 2

Docentes:

- Ing. Sebastián Balestrini
- Esp. Ing. Gustavo Di Stefano



Objetivos

- Aprender a escribir algoritmos en pseudocódigo.
- Aplicar estructuras de control (decisión y repetición).
- Consolidar la lógica algorítmica antes de programar en C.



Herramientas

- Diseño y ejecución paso a paso un diagrama de flujo para ver cómo fluyen los datos y entender la lógica. https://flowrun.io

- Para mejor legibilidad del pseudocódigo se puede usar Notepad++ (o similares) utilizando el formato del lenguaje C. https://notepad-plus-plus.org/downloads

Variables contadoras y sumadoras

¿Que son?

- Las variables contadoras y sumadoras son variables auxiliares que se utilizan dentro de estructuras repetitivas para:
- **Sumadora**: acumular un resultado (por ejemplo, la suma de varios valores).
- **Contadora**: llevar la cuenta de cuántas veces ocurre una acción.

```
suma = suma + valor
```

```
cuenta = cuenta + 1
```

Son fundamentales para procesar datos en bucles y permiten calcular totales, promedios, frecuencias, etc.

Pseudocódigo

¿Qué es una estructura repetitiva?

- Permite ejecutar varias veces un mismo conjunto de instrucciones.
- También se llaman "bucles" o "ciclos"

¿Cuando se usan?

- Se usan cuando hay tareas repetitivas o acumulativas.

Ejemplo:

```
1  inicio()
2  {
3     // Imprimir de 0 a 4
4     para (i = 0; i < 5; i++)
5     {
6        imprimir(i);
7     }
8  }
```

```
10 Salida
11
12 0  1  2  3  4
13
```


FADENA
FACULTAD DE DEFENSA NACIONAL

Tipos de estructuras repetitivas

- Repetitiva exacta: Ejecutan el ciclo un numero conocido de veces, llamados ciclos "***para***" (for)

```
para (i = 0; i < 10; i++)
{
    // instrucciones repetidas
}
```

- Repetitiva Inexacta (0 a n): *la condición* para evaluar la cantidad de repeticiones esta *al comienzo* del bloque.

```
i = 0;
mientras (i < 10)
{
    // instrucciones repetidas
    i = i +1;
}
```

- Repetitiva Inexacta (1 a n): *la condición* para evaluar la cantidad de repeticiones esta *al final* del bloque.

```
i = 0;
hacer
{
    // instrucciones repetidas
    i = i +1;
} mientras(i < 10)
```

Ciclo “para” (for)

Ejemplo 1: Imprimir por pantalla los números de 0 a 4.

```
1  inicio()
2  {
3      para (i = 0; i < 5; i++)
4      {
5          imprimir(i);
6      }
7  }
8
```

```
16 Resultado
17
18 0 1 2 3 4
```

Ejemplo 2: Sumar los 5 primeros números.

```
1  inicio()
2  {
3      suma = 0;
4
5      para (i = 1; i <= 5; i++)
6      {
7          suma = suma + i;
8      }
9
10     imprimir("La suma es:", suma);
11 }
12
```

```
16 Resultado
17
18 La suma es: 15
```


FADENA
FACULTAD DE DEFENSA NACIONAL

Ciclo “mientras” (while)

Ejemplo 3: Imprimir por pantalla los números de 0 a 4.

```
inicio()
{
    i = 0;
    mientras (i < 5)
    {
        imprimir(i);
        i = i + 1;
    }
}
```

```
Resultado

0 1 2 3 4
```

Ejemplo 4: Leer números hasta ingresar un valor negativo y mostrar la suma total

```
inicio()
{
    suma = 0;
    ingresar(num);

    mientras (num >= 0)
    {
        suma = suma + num;
        ingresar(num);
    }

    imprimir("La suma total es:", suma);
}
```

Ciclo hacer – mientras (do - while)

Ejemplo 5: Imprimir por pantalla los números de 0 a 4.

(Notar que al menos el ciclo se ejecuta una vez)

```
inicio()
{
    i = 0;
    hacer
    {
        imprimir(i);
        i = i + 1;
    } mientras (i < 5);
}
```

```
Resultado

0 1 2 3 4
```

Ejemplo 6: Validar una contraseña.

```
inicio()
{
    hacer
    {
        imprimir("Ingresar contraseña:");
        ingresar(clave);
    } mientras (clave != "1234");

    imprimir("Acceso concedido");
}
```

Que aprendimos hasta aca ?

- Qué son los ciclos for, while y do while.
- Cómo usar variables contadoras y sumadoras.
- Cuándo usar cada tipo de ciclo.
- Estamos listos para comenzar a codificar.



Próximo paso

A partir de la próxima clase, comenzamos a ver cómo esto se transforma en código real en lenguaje C

¿Preguntas o dudas?



FADENA
FACULTAD DE DEFENSA NACIONAL